Project Assignment 1

Compiler Design Lab
Prof. Dr. Sebastian Hack
Fabian Ritter, B.Sc.

Project Task A General Information

In the practical project, you will implement a compiler for a subset of C. The project is composed of several milestones that roughly reflect the structure of a compiler, following the lectures. Each milestone comes with a soft deadline that indicates the expected time schedule. The project milestones are not graded at the soft deadlines, only the final submission at the end of the semester will be graded. The project grade is based on the percentage of tests passed per project stage and a code review where the group members have to justify their work. Each member of a group needs to contribute a substantial part to the project submission to pass the project.

During the project phases, you will receive feedback that hints at existing shortcomings in your compiler in the form of daily test runs on our server. We strongly suggest that you create your own test suite for the project and not rely only on the feedback from our test runs. While the compiler itself is a graded work that each group has to perform individually, you are encouraged to exchange test cases with other project groups in the repository at `https://cc.cdl.uni-saarland.de/student-tests-2022/testsuite`.

Technical requirements and restrictions:

- The compiler itself **must** be written in C++.
- Your project must be buildable with the provided Makefile, which produces a binary `c4`.
- Upon acceptance of a source program, `c4` must terminate with return code 0. Rejection of a source program must be indicated by return code 1 and an error message with the following format on the `stderr` output stream:

  `<filename>:<line>:<column>: error: <message>`

  Here, `<filename>` is the name of the input file as provided to your compiler, `<line>` and `<column>` are the line number and the column number of the error's location, and `<message>` is some meaningful error message.
- The individual assignments will refine the requirements and specify the subset of C that we consider for this project. If you feel like it, you can implement more than this subset in your compiler. Generally, we require you to accept and correctly compile all programs that are part of the specified subset and to reject all programs that are not a part of the full C language. Programs that are not in those two classes, i.e. that are valid in C but not part of our subset, you may either reject with an error message or implement correctly according to the C language specification.

To get started:

- Make sure that you have followed the "First Steps" at `https://cms.sic.saarland/cc22/4/How_to_Attend`. You should then have a Gitlab account, be in a group there with your team mates and you should have a fresh project with the name `c4` (for "Compiler Construction C Compiler", C[4]) and default branch `main` living in your group's namespace.
- When in your project's root folder, merge the template project into your sources:

```
git remote add template git@cc.cdl.uni-saarland.de:template/c4.git
git fetch --all
git merge --allow-unrelated-histories template/main
```

- Examine the contents of the starter kit. It provides a Makefile that you can use to build the project. The produced binary is called `c4` and sits in a subdirectory of the build directory. The name of the subdirectory is based on the build configuration that you use (`debug` by default, other configurations can be chosen by setting the `CFG` environment variable to one of the configurations in the 'config' subdirectory). The server tests will use the `debug` configuration. Additionally, the `impl_ideas` directory contains code that you may use and adjust to implement your compiler. Consider its `README.md` for more details. For now, you can ignore the `build_llvm.sh` script.
- Get the C11 language specification from `http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf`.
- You can now begin to work with your new project.

Project Task B Lexer

Implement the lexical analysis. The relevant section in the standard is §6.4. The lexer uses the maximal munch strategy (§6.4p4). The project does not incorporate a preprocessor. Therefore we are only considering *token*s, not *preprocessing-token*s. Furthermore, *universal-character-name*s (§6.4.3) are omitted. For *constant*s (§6.4.4) we process only *decimal-constant*s, 0 and *character-constant*s. In this connection we ignore *integer-suffix*es and prefixes for the encoding (`L`, ...). *character-constant*s are restricted to single characters. For *string-literal*s (§6.4.5) the *encoding-prefix* is also omitted. Also we elide *octal-escape-sequence*s and *hexadecimal-escape-sequence*s.

The phases of translation are presented in §5.1.1.2. We leave out several parts. In phase 1, trigraphs are elided. For the input and execution encodings we use ISO-8859-1. Phases 2, 4 and 6 are left out. It is not necessary to implement the remaining phases in detail. Some of them may be combined.

To test the lexer, output the tokens. For this, use the switch −−tokenize, e.g. c4 −−tokenize test.c. Output one token per line: First the position in the source (terminated by a colon), a single space, then the kind of token, another single space, and, last, the textual representation of the token. The position is a tuple of line and column in the source file, both counting from 1. New lines are started by a CR LF sequence ("\r\n"), or by CR or LF characters ("\r" and "\n") if those are not part of a CR LF sequence. Make sure to output string- or character-constants exactly like they appear in the input stream, so do not replace escape sequences in the output. Similarly, do not replace digraphs with their more common counterparts. For naming the tokens, use the non-terminals on the right hand side of the productions of *token* (§6.4p1). Consider the example input file test.c:

```
42␣␣if
␣␣"bla\n"x+
```

Output:

```
test.c:1:1: constant 42
test.c:1:5: keyword if
test.c:2:2: string-literal "bla\n"
test.c:2:9: identifier x
test.c:2:10: punctuator +
```

Files that contain lexical errors must be rejected with an error message showing the location of the error. For error locations, choose the location of the first character that makes the input invalid.[1]

The running time of your lexer must be proportional to the input size.

Remarks and hints:

- Write more test cases, especially some that test the rejection capabilities of your lexer.
- §6.4.2.1p4 suggests a neat way for the implementation of the detection of keywords.
- Do not convert the number tokens to integers in the lexer. If you do, the lexer becomes dependent on the integer size of the machine your compiler runs on, which you generally want to avoid.
- Do not intermingle lexing with the output of the tokens. The interface to read tokens will be used again by the parser.
- It will prove helpful in the following phase to end the token stream of your lexer with a (not printed) end-of-file token.
- You may also use an existing lexer generator to create (parts of) the source code for this task.
- The soft deadline for this milestone is 2022-11-09.
- Keep it simple!

[1] While this is not necessarily the most helpful error location for developers using the compiler, it is better for automated testing.